Student Record Search System Using Binary Search Tree (BST)

Name: Anjali Kumari **Roll no:** 2556427 **Branch:** BTech CSE B1
Sem: 3rd

Project Description

The Student Record Search System is a menu-driven C program that manages student records using a Binary Search Tree (BST). Each student record contains an enrolment ID, student name, and department.

The system allows users to insert, search, delete, and display student records. It uses the BST data structure to organize records according to enrolment IDs, making searching and sorting easier.

Features

- **Insert Student:** Add a new student record with a unique enrolment ID, name, and department.
- **Search Student:** Search for a student using their enrolment ID.
- **Delete Student:** Remove a student record from the BST.
- **Inorder Traversal:** Display student records in ascending order of enrolment ID.
- **Preorder Traversal:** Display the tree in preorder sequence.
- **Postorder Traversal:** Display the tree in postorder sequence.
- **Duplicate Handling:** Prevent insertion of records with duplicate enrolment IDs.
- **Sample Records:** Load 10 predefined student records for testing.

Technologies Used

- **Programming Language:** C
- **Libraries:**
    - stdio.h – Input and output operations.
    - string.h – String handling.
    - stdlib.h – Dynamic memory allocation and memory management.

Data Structure Used

Binary Search Tree (BST)

A Binary Search Tree is a non-linear data structure in which each node contains a key and two possible child nodes.

- The left child contains a smaller enrolment ID.
- The right child contains a greater enrolment ID.
- Duplicate enrolment IDs are not allowed.

Each node stores:

- Enrolment ID
- Student Name
- Department
- Pointer to the left child
- Pointer to the right child
-

Tree Traversal Methods

1. **Inorder:** Left → Root → Right
2. **Preorder:** Root → Left → Right
3. **Postorder:** Left → Right → Root

Inorder traversal displays student records in ascending order of enrolment ID.

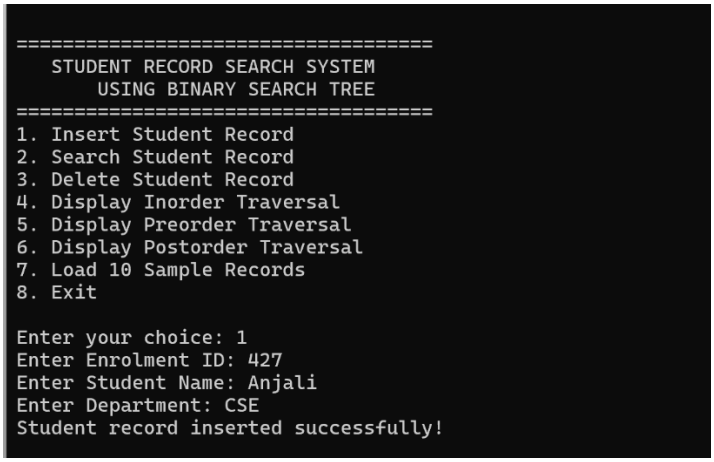
Time Complexity

Operation	Average Case	Worst Case
Insertion	O(log n)	O(n)
Searching	O(log n)	O(n)
Deletion	O(log n)	O(n)
Traversal	O(n)	O(n)

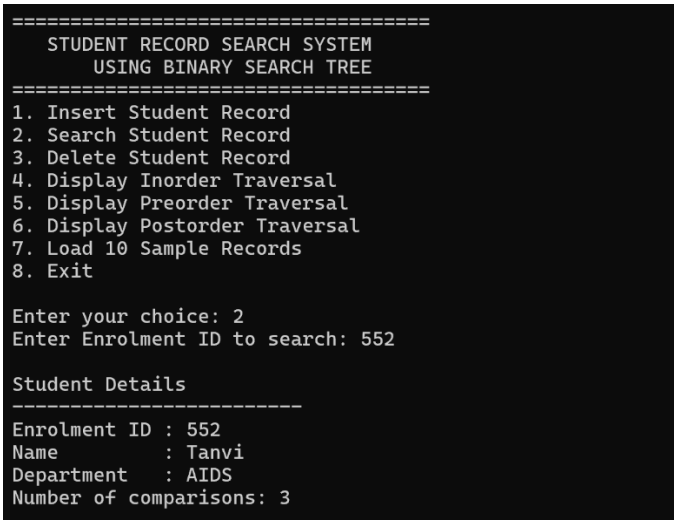
A regular BST may become skewed if records are inserted in sorted order, resulting in O(n) operations. Self-balancing trees such as AVL trees help maintain efficient operations.

Sample Operations

1. Insert Student Record



2. Search Student Record



3. Delete Student Record

```
====================================
   STUDENT RECORD SEARCH SYSTEM
       USING BINARY SEARCH TREE
====================================
1. Insert Student Record
2. Search Student Record
3. Delete Student Record
4. Display Inorder Traversal
5. Display Preorder Traversal
6. Display Postorder Traversal
7. Load 10 Sample Records
8. Exit

Enter your choice: 3
Enter Enrolment ID to delete: 552
Student record deleted successfully!
```

4. Display Inorder Traversal

```
====================================
   STUDENT RECORD SEARCH SYSTEM
       USING BINARY SEARCH TREE
====================================
1. Insert Student Record
2. Search Student Record
3. Delete Student Record
4. Display Inorder Traversal
5. Display Preorder Traversal
6. Display Postorder Traversal
7. Load 10 Sample Records
8. Exit

Enter your choice: 4

STUDENT RECORDS (SORTED BY ID)
------------------------------------------------
ID         NAME                DEPARTMENT
101        Ananya              CSE
102        Sneha               IT
103        Priya               IT
104        Rohan               IT
105        Aarav               CSE
106        Neha                ECE
107        Rahul               ECE
108        Arjun               ECE
109        Karan               CSE
110        Meera               CSE
427        Anjali              CSE
504        Swati               CSE
```

5. Display Preorder Traversal

```
====================================
   STUDENT RECORD SEARCH SYSTEM
       USING BINARY SEARCH TREE
====================================
1. Insert Student Record
2. Search Student Record
3. Delete Student Record
4. Display Inorder Traversal
5. Display Preorder Traversal
6. Display Postorder Traversal
7. Load 10 Sample Records
8. Exit

Enter your choice: 5

Preorder Traversal: 427 105 103 101 102 104 107 106 109 108 110 504
```

6. Display Postorder Traversal

```
==================================
   STUDENT RECORD SEARCH SYSTEM
       USING BINARY SEARCH TREE
==================================
1. Insert Student Record
2. Search Student Record
3. Delete Student Record
4. Display Inorder Traversal
5. Display Preorder Traversal
6. Display Postorder Traversal
7. Load 10 Sample Records
8. Exit

Enter your choice: 6

Postorder Traversal: 102 101 104 103 106 108 110 109 107 105 504 427
```

7. Load 10 Sample Records

```
==================================
   STUDENT RECORD SEARCH SYSTEM
       USING BINARY SEARCH TREE
==================================
1. Insert Student Record
2. Search Student Record
3. Delete Student Record
4. Display Inorder Traversal
5. Display Preorder Traversal
6. Display Postorder Traversal
7. Load 10 Sample Records
8. Exit

Enter your choice: 7
Student record inserted successfully!
Student record inserted successfully!
Student record inserted successfully!
Student record inserted successfully!
Student record inserted successfully!
Student record inserted successfully!
Student record inserted successfully!
Student record inserted successfully!
Student record inserted successfully!
Student record inserted successfully!

10 sample records loaded successfully!
```

8. Checking Duplicates

```
==================================
   STUDENT RECORD SEARCH SYSTEM
       USING BINARY SEARCH TREE
==================================
1. Insert Student Record
2. Search Student Record
3. Delete Student Record
4. Display Inorder Traversal
5. Display Preorder Traversal
6. Display Postorder Traversal
7. Load 10 Sample Records
8. Exit

Enter your choice: 1
Enter Enrolment ID: 427
Enter Student Name: Anjali
Enter Department: CSE
Duplicate enrolment ID! Record not inserted.
```

Concepts Covered

- Binary Trees and Binary Search Trees
- Structures and Pointers
- Dynamic Memory Allocation using malloc()
- Memory Deallocation using free()

- Recursion
- Insertion, Searching and Deletion
- Tree Traversals
- Time and Space Complexity

BST vs AVL Tree

A BST can have O(n) complexity: When elements are inserted in increasing order (1, 2, 3, ...), each new node is added to the right side of the previous node. This creates a one-sided tree that looks like a linked list(Skewed tree). Since the height becomes n, searching, insertion, and deletion may require checking up to n nodes.

AVL trees solve this problem: An AVL tree is a self-balancing form of a Binary Search Tree. It maintains the height difference between the left and right subtrees of every node within -1, 0, or +1. This difference is called the balance factor. Whenever insertion or deletion disturbs the balance, the tree uses rotations (LL, RR, LR, and RL) to bring it back to balance. This keeps the tree height at O(log n), ensuring that searching, insertion, and deletion take O(log n) time even in the worst case.

Conclusion

The Student Record Search System demonstrates how a Binary Search Tree can be used to store and manage student information efficiently. It implements insertion, searching, deletion, and different traversal techniques while demonstrating important concepts of data structures, recursion, pointers, and dynamic memory management.